

Orchestration and resilience

Step Functions, Amazon MWAA, EventBridge, Glue workflows, and what happens when a pipeline fails at three in the morning.

Orchestration is not about running steps. It is about **what happens when one of them does not work.**

The four ways to orchestrate

TOOL	CHOOSE IT WHEN
AWS Step Functions	Coordinating AWS services, with visual state and built-in error handling
Amazon MWAA	Apache Airflow. Existing DAGs, Python authoring, or a hybrid estate
Glue workflows	The pipeline is entirely Glue jobs, crawlers and triggers
EventBridge	Something happened, or it is time. Triggering rather than coordinating

ON THE EXAM

EventBridge starts work. The other three coordinate it. Mixing those up is the most common error here.

AWS Step Functions, in the parts that matter

- ◆ **A state machine** defined in JSON, with each state a task, choice, wait or parallel branch.
- ◆ **Optimised integrations** call Glue, EMR, Athena, Lambda and more without code.
- ◆ **Retry and Catch** are declared per state, not written into your job.
- ◆ **Visual execution history** shows exactly which state failed and with what input.
- ◆ **Distributed Map** fans out across millions of S3 objects in parallel.

ON THE EXAM

Retry and error handling are configuration, not code. That is the main argument for Step Functions in exam answers.

Fanning out over very large collections

- ◆ **Inline Map** runs the child steps within the parent execution. Limited concurrency.
- ◆ **Distributed Map** runs each item as its own child execution. Very high concurrency.
- ◆ **Reads the item list from S3 directly**, including a manifest or a CSV.
- ◆ **Set a maximum concurrency** to protect whatever the children write to.
- ◆ **Batch items** so one child handles many, rather than one child per tiny object.

ON THE EXAM

Millions of objects to process independently is the Distributed Map signal. Inline Map runs out well before that.

The state types

STATE	WHAT IT DOES
Task	Does the work. Calls a service or a Lambda function
Choice	Branches on a condition in the data
Wait	Pauses for a duration, or until a timestamp
Parallel	Runs several branches at once, waits for all
Map	Runs the same steps over each item in a collection
Pass, Succeed, Fail	Shape the data, or end the execution deliberately

ON THE EXAM

Choice and Wait are why a state machine beats a script. Conditional logic and waiting are first-class, not code.

Three ways to call a service

PATTERN	BEHAVIOUR
Request response	Call it and move on. Does not wait for the work to finish
Run a job, the .sync pattern	Call it and wait for completion. Reports the real outcome
Wait for callback	Pause until something sends a token back, up to a year

ON THE EXAM

The default is request-response, which starts a Glue job and immediately calls the step successful. That is rarely what you want.

Passing data between steps

- ◆ **Each state receives JSON** and passes JSON to the next one.
- ◆ **ResultPath** decides whether output replaces the input or sits beside it.
- ◆ **Intrinsic functions** do simple manipulation without a Lambda function.
- ◆ **There is a payload size limit.** Large data belongs in S3, with the path passed along.
- ◆ **Glue workflow run properties** do the equivalent job inside Glue workflows.

THE TIEBREAKER

Never pass a dataset through a state machine. Pass the location of the dataset.

Orchestrating each engine

TO RUN THIS	STEP FUNCTIONS DOES THIS
An AWS Glue job	StartJobRun with .sync, so it waits for the job to finish
An Amazon EMR step	Adds a step to a running cluster, or creates a transient one
An Amazon Athena query	StartQueryExecution with .sync, then reads the result location
Amazon Redshift SQL	The Redshift Data API, which needs no persistent connection
An AWS Lambda function	Direct invocation, synchronous by default
A human decision	Wait for callback with a task token, for up to a year

ON THE EXAM

Step Functions can create and terminate an EMR cluster as part of the workflow. That is how transient clusters get orchestrated.

Standard or Express workflows?

STANDARD

Long and auditable

- Runs for up to one year
- Full execution history retained
- Exactly-once execution
- Billed per state transition

EXPRESS

Short and high volume

- Five minute maximum duration
- History goes to CloudWatch Logs
- At-least-once execution
- Billed by duration and memory

THE TIEBREAKER

Data pipelines are almost always Standard. Express is for high-volume event processing measured in seconds.

Amazon MWAA, and what changed

- ◆ **Managed Apache Airflow.** Python DAGs, the full operator ecosystem.
- ◆ **Provisioned environments** run a scheduler and database continuously, billed by the hour.
- ◆ **MWAA Serverless** arrived in November 2025. Pay per task, no environment.
- ◆ **Airflow 3** is supported, alongside the 2.x line.
- ◆ Serverless bills per second per task, with a one minute minimum per task.

ON THE EXAM

The old objection to MWAA was paying for an idle environment. Serverless removes it, and older material predates that.

Airflow vocabulary

TERM	MEANING
DAG	The workflow. A graph of tasks with dependencies and no cycles
Operator	A task template. There are operators for most AWS services
Sensor	A task that waits for a condition, such as a file appearing
XCom	A small value passed between tasks
Backfill	Running a DAG for past intervals it has not run for
Catchup	Whether a newly enabled DAG runs all missed intervals automatically

ON THE EXAM

Catchup set to true on a DAG with two years of history will try to run two years of workflows at once.

Waiting without wasting

- ◆ A **naive sensor** occupies a worker slot for the whole wait, doing nothing.
- ◆ **Deferrable operators** release the worker and resume when the condition is met.
- ◆ **Step Functions Wait states** consume no compute at all while waiting.
- ◆ A **Lambda function that sleeps** is billed for every second of the sleep.
- ◆ **Always set a timeout**, so a wait for something that never arrives eventually fails.

ON THE EXAM

Waiting is a first-class operation in an orchestrator. Implementing it inside a job is what costs money.

Step Functions or MWAA?

STEP FUNCTIONS

AWS-native coordination

- Defined in JSON or the visual editor
- Deep service integrations, no code
- Nothing to run between executions
- Weaker at complex scheduling logic

AMAZON MWAA

Airflow

- Existing DAGs run unchanged
- Python authoring, huge operator library
- Orchestrates outside AWS too
- More concepts to learn and operate

THE TIEBREAKER

Existing Airflow DAGs or a hybrid estate means MWAA. A greenfield AWS-only pipeline means Step Functions.

Glue workflows, and their ceiling

- ◆ **Chains Glue jobs, crawlers and triggers** with conditional logic on success or failure.
- ◆ **No extra service.** It is part of Glue, so nothing new to learn or pay for.
- ◆ **Workflow run properties** pass values between jobs in the chain.
- ◆ **Glue only.** A Lambda function or a Redshift statement is outside its reach.
- ◆ **No human approval, no branching on arbitrary conditions.**

THE TIEBREAKER

All-Glue pipeline means Glue workflows. One non-Glue step means Step Functions.

Orchestration inside SageMaker Unified Studio

- ◆ **Unified Studio includes its own workflows**, built on Apache Airflow.
- ◆ Lets a project define pipelines alongside its catalog, notebooks and queries.
- ◆ **Aimed at project-scoped work** rather than platform-wide orchestration.
- ◆ Part of the SageMaker material added to the exam guide in version 1.1.
- ◆ Not a replacement for Step Functions or standalone MWAA.

ON THE EXAM

New to the exam guide. Module twenty covers Unified Studio domains, projects and governance in full.

Every pipeline starts because of **a clock or an event**. Questions almost always say which.

Ways to start work

MECHANISM	WHAT IT IS FOR
EventBridge Scheduler	One-off or recurring schedules at scale. Cron or rate expressions
EventBridge rules	React to an AWS service event, or a custom event you publish
S3 Event Notifications	React directly to object creation, removal or restore
Glue triggers	Schedule, on demand, or conditional on another Glue job
Airflow schedule	A DAG's own schedule interval, inside MWAA
Direct invocation	An SDK call from an application or another service

ON THE EXAM

EventBridge Scheduler is the newer, purpose-built scheduler. EventBridge rules are for reacting to events.

S3 notifications or EventBridge?

S3 EVENT NOTIFICATIONS

Direct and simple

- Configured on the bucket itself
- Targets SNS, SQS or Lambda
- Filter by prefix and suffix only
- Fewer moving parts

EVENTBRIDGE FROM S3

Richer routing

- Events go to an event bus
- Many targets, including Step Functions
- Filter on any field in the event
- Archive and replay events

THE TIEBREAKER

More than one consumer, or filtering on anything beyond prefix and suffix, means EventBridge.

A company runs a nightly pipeline consisting of four AWS Glue jobs that must run in order, a Glue crawler after the second job, and a final AWS Lambda function that posts a summary to a Slack channel. The pipeline is currently started by a cron schedule on each job, with each job's schedule set far enough apart that the previous one should have finished.

Occasionally an early job runs long, and a later job starts against incomplete data without any error being raised. The team wants the pipeline to be dependency-driven and to alert on failure.

What should the data engineer implement?

- ☐ A Move all five steps into an AWS Glue workflow with conditional triggers
- ☐ B Build an AWS Step Functions state machine invoking each step in sequence, started by EventBridge Scheduler
- ☐ C Increase the gaps between the cron schedules to allow more headroom
- ☐ D Set the maximum concurrent runs on each Glue job to one

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 Glue workflow
Right instinct, and the Lambda function is not a Glue component. Glue workflows cannot orchestrate it.
- B

 Step Functions started by EventBridge Scheduler
Each step starts only when the previous one succeeded, and a Catch on any state can raise an alert.
- C

 Wider cron gaps
Guessing at timing more conservatively. The same failure returns on a slower night, and the pipeline finishes later every night regardless.
- D

 Max concurrent runs of one
Stops a job overlapping itself. It does nothing about a different job starting on incomplete data.

THE SIGNAL

One Lambda function in an otherwise all-Glue pipeline is what rules out Glue workflows. Count the non-Glue steps.

A resilient pipeline is not one that never fails. It is one that fails **loudly, safely and recoverably.**

The four mechanisms

- ◆ **Retry with exponential backoff** for transient failures. Throttling, timeouts, brief outages.
- ◆ **A dead-letter queue** for messages that will never succeed, so they are kept, not lost.
- ◆ **Catch and route** so a failed state moves to handling rather than stopping dead.
- ◆ **Idempotency**, so a retry produces the same result as the first attempt.
- ◆ Retries without idempotency create duplicates. Module two showed exactly that.

THE TIEBREAKER

Retry is for failures that might succeed next time. A dead-letter queue is for failures that will not.

Exponential backoff, and why it is not just politeness

- ◆ **Immediate retries** against a throttling service make the throttling worse.
- ◆ **Exponential backoff** doubles the wait between attempts, giving the source room to recover.
- ◆ **Jitter** randomises the wait so retrying clients do not synchronise into a spike.
- ◆ **Step Functions Retry** takes an interval, a backoff rate and a maximum attempt count.
- ◆ **AWS SDKs retry automatically**, which can hide the problem from your logs.

THE TIEBREAKER

Throttling questions want backoff, not a higher retry count. Retrying harder is the wrong direction.

SNS or SQS?

AMAZON SNS

Tell everyone

- Publish once, many subscribers receive it
- Push. Subscribers do not poll
- No storage. A missing subscriber misses out
- Fan-out, alerts, notifications

AMAZON SQS

Hand off work

- One consumer takes each message
- Pull. Consumers read at their own pace
- Messages persist until consumed
- Buffering, decoupling, retry with a DLQ

THE TIEBREAKER

SNS distributes news. SQS distributes work. A very common pattern uses SNS fanning out into several SQS queues.

Queue mechanics worth knowing

- ◆ **Visibility timeout.** How long a message is hidden after a consumer takes it.
- ◆ **Too short** and a slow consumer's message reappears and is processed twice.
- ◆ **Dead-letter queue** after a set number of failed receives.
- ◆ **Standard queues** are at-least-once, best-effort ordering. **FIFO** guarantees both.
- ◆ **Long polling** reduces empty receives and cost.

ON THE EXAM

Duplicate processing with no code changes usually means the visibility timeout is shorter than the processing time.

A pipeline reads messages from an Amazon SQS queue with an AWS Lambda function that calls a partner API for each message. The partner API is rate limited and returns throttling errors during busy periods.

The team has configured the Lambda function with ten retries and no delay between them. During busy periods the queue backs up, throttling errors increase sharply, and some messages eventually exceed the maximum receive count and are lost because no dead-letter queue is configured.

Which combination of changes best addresses the problem?

- ☐ A Increase the Lambda retry count from ten to thirty
- ☐ B Add exponential backoff with jitter to the API calls, and configure a dead-letter queue on the SQS queue
- ☐ C Increase the Lambda function's reserved concurrency so more messages are processed in parallel
- ☐ D Switch the SQS queue from standard to FIFO to control the processing order

Which combination of changes best addresses the problem?

- A

 Thirty retries instead of ten
Three times the load on a service already refusing requests. Retrying harder is the wrong direction.
- B

 Backoff with jitter, plus a dead-letter queue
Backoff lets the partner recover; jitter stops synchronised retries; the DLQ means nothing is lost.
- C

 More concurrency
Sends requests faster to an API that is already rate limiting. This makes throttling worse, not better.
- D

 FIFO queue
Guarantees ordering, which nothing here asked for. It also lowers throughput and does not touch the throttling.

THE SIGNAL

Two of the four options increase pressure on a rate-limited service. That pattern is the exam checking your instinct.

Designing for the failure you will actually get

- ◆ **Fail fast on bad data.** A pipeline that continues on partial data is worse than one that stops.
- ◆ **Make each step idempotent** so a rerun is always safe.
- ◆ **Checkpoint long pipelines** so a rerun resumes rather than restarting.
- ◆ **Alert on absence**, not only on errors. A job that never ran raises no error.
- ◆ **Separate transient from permanent** failures and handle each differently.

ON THE EXAM

The hardest failure to detect is the job that quietly did not run at all. Nothing errors, and nobody is told.

Reruns and backfills

- ◆ **Parameterise the date** so a run can be told which day to process.
- ◆ **A pipeline hardcoded to yesterday** cannot reprocess last Tuesday.
- ◆ **Overwrite the target partition** so a rerun replaces rather than duplicates.
- ◆ **Glue bookmarks can be reset** to reprocess from a point in time.
- ◆ **Bound the concurrency of a backfill**, or it overwhelms the target.

ON THE EXAM

The question "can you rerun last month?" separates a production pipeline from a script that happens to run daily.

A daily pipeline loads sales data into Amazon Redshift. If the source file has not arrived by six in the morning, the pipeline currently proceeds anyway with whatever partial data is present, and the daily report is produced from incomplete figures.

Finance has twice made decisions on incorrect numbers without knowing they were incomplete. The team wants the pipeline to handle a late or missing file correctly.

What is the most appropriate design?

- ☐ A Run the pipeline later in the morning so the file is more likely to have arrived
- ☐ B Make the pipeline event-driven on file arrival, with a Step Functions wait and an alert if the file has not arrived by a deadline
- ☐ C Load the partial data but add a note to the report saying it may be incomplete
- ☐ D Retry the load three times with exponential backoff before continuing

◆ PAUSE AND DECIDE

What is the most appropriate design?

- A

 Run later
Guessing at a later time. Same failure on a later day, and the report is delayed every single day for a rare event.
- B

 Event-driven with a wait and a deadline alert
The pipeline starts when the data exists, and a missing file raises an alert instead of producing wrong numbers.
- C

 Publish partial data with a caveat
Still produces incorrect figures, now with a disclaimer nobody reads. It does not solve the stated harm.
- D

 Retry with backoff
Backoff is for transient failures. A file that has not been produced yet will not appear because you retried.

THE SIGNAL

Compare with scenario 2. There backoff was right because the failure was transient. Here it is wrong because nothing is failing.

Making pipelines scale

- ◆ **Decouple with a queue** so a slow consumer does not block a fast producer.
- ◆ **Parallelise independent work.** Step Functions Map, or parallel branches.
- ◆ **Distributed Map** for very large fan-out across S3 objects.
- ◆ **Watch downstream limits.** Parallelism upstream can throttle a database downstream.
- ◆ **Reserved concurrency on Lambda** caps how hard a function hits a fragile dependency.

ON THE EXAM

Reserved concurrency is a throttle you apply to yourself, protecting whatever sits behind the function.

Availability and recovery

- ◆ **Managed services are multi-AZ already.** S3, Glue, Step Functions, Lambda.
- ◆ **What you own is what you design for.** EMR clusters and MSK live in your subnets.
- ◆ **Keep source data,** so any derived dataset can be rebuilt from it.
- ◆ **Define pipelines as code,** so a Region can be rebuilt rather than restored.
- ◆ **Cross-Region replication** for the raw layer if the recovery objective demands it.

ON THE EXAM

The best recovery story for a derived dataset is that you still have the raw data and the code to rebuild it.

Across accounts and Regions

- ◆ A state machine can assume a role in another account to run a step there.
- ◆ EventBridge buses can forward events to a bus in another account or Region.
- ◆ Cross-Region adds latency and data transfer cost to every step.
- ◆ Keep orchestration close to the data it coordinates, not centralised for tidiness.
- ◆ Central orchestration, distributed execution is the usual pattern at scale.

ON THE EXAM

A central account can coordinate pipelines that execute in the accounts owning the data.

What orchestration costs

TOOL	BILLING
Step Functions Standard	Per state transition. A simple pipeline costs almost nothing
Step Functions Express	By duration and memory, like Lambda
MWAA provisioned	Per hour for the environment, whether DAGs run or not
MWAA Serverless	Per task, per second, one minute minimum per task
Glue workflows	No charge for the workflow. You pay for the jobs it runs
EventBridge	Per million events. Effectively free at pipeline volumes

ON THE EXAM

Glue workflows and EventBridge add no orchestration cost at all. That is a real argument when they fit.

An orchestration team runs about forty Apache Airflow DAGs on a self-managed Airflow deployment on Amazon EC2. The DAGs use a mix of AWS operators and operators for two on-premises systems. Roughly half the DAGs run once a week; the rest run daily and complete in minutes.

The team spends significant time patching Airflow, upgrading the metadata database and recovering the scheduler. They want to reduce that work and stop paying for capacity that is idle most of the time. The DAGs must keep running.

What should the team do?

- ☐ A Rewrite the DAGs as AWS Step Functions state machines
- ☐ B Migrate to Amazon MWAA Serverless, keeping the existing Airflow workflows
- ☐ C Migrate to a provisioned Amazon MWAA environment sized for peak load
- ☐ D Move the Airflow deployment to containers on Amazon EKS with cluster autoscaling

◆ PAUSE AND DECIDE

What should the team do?

A Rewrite as Step Functions

Forty DAGs rewritten, and Step Functions does not reach the on-premises systems those operators handle.

C Provisioned MWAA

Removes the patching, and keeps paying for an environment that is idle most of the week. Half the requirement.

B MWAA Serverless

Airflow workflows keep running, AWS operates the scheduler and database, and idle time costs nothing.

D Airflow on EKS

Trades one set of operational work for another. You now patch Airflow and operate Kubernetes.

THE SIGNAL

Two requirements: less operational work, and no payment for idle capacity. Only one option satisfies both.

Watching an orchestrator

- ◆ **Step Functions publishes metrics** for executions started, succeeded, failed and timed out.
- ◆ **The execution history** shows every state, its input, its output and each retry.
- ◆ **EventBridge can react to a failed execution** and notify through SNS.
- ◆ **MWAA writes scheduler, worker and task logs** to CloudWatch Logs.
- ◆ **Alarm on executions not started**, to detect a trigger that stopped firing.

THE TIEBREAKER

Failure notification belongs outside the pipeline. A pipeline that has died cannot send its own alert.

Keeping workflows manageable

- ◆ **Nested state machines.** A step can start another workflow and wait for it.
- ◆ Lets teams own their own pipelines while a parent coordinates them.
- ◆ Keeps a single definition readable rather than a hundred states long.
- ◆ **In Airflow**, the equivalent is one DAG triggering another.
- ◆ Watch for circular dependencies between workflows that wait on each other.

ON THE EXAM

A very large single workflow is a maintenance problem. Composition is the usual exam-correct answer.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
A step starts on incomplete data	Time-based coordination instead of dependency-based
The same work happens twice	Visibility timeout too short, or a retry over a non-idempotent step
Throttling gets worse under load	Retries without backoff, amplifying the load
Messages disappear after repeated failures	No dead-letter queue configured
Nothing ran and nobody noticed	Alerting on failure only, not on absence
A trigger fires but the job is skipped	Glue max concurrent runs reached; the run is rejected, not queued

ON THE EXAM

The last row is from module two. An event-driven Glue job with the default concurrency of one drops overlapping triggers.

A Step Functions state machine processes files by invoking a Lambda function per file. It has run reliably for a year. Last week the upstream system began sending a small number of files with a malformed header.

The state machine now stops on the first malformed file. Every valid file behind it in that execution is left unprocessed, and the team discovers the problem the following morning. Retries are configured with three attempts and backoff.

What is the most appropriate change?

- ☐ A Increase the retry attempts from three to ten so the file eventually succeeds
- ☐ B Add a Catch on the state that routes failed files to a dead-letter location and continues processing the rest
- ☐ C Remove the retry configuration so the state machine fails faster
- ☐ D Switch the state machine from Standard to Express workflows

◆ PAUSE AND DECIDE

What is the most appropriate change?

A Ten retries

A malformed header fails identically every time. Ten attempts produce ten identical failures, more slowly.

C Remove retries

Fails faster and still stops the whole execution. It also removes protection against genuine transient failures.

B Catch, route the bad file aside, continue

Separates a permanent failure from a transient one. Bad files are kept for investigation; good files still get processed.

D Express workflows

Changes duration limits and pricing. It has nothing to do with how a failed state is handled.

THE SIGNAL

A failure that will never succeed needs routing, not retrying. This is the distinction from the mechanisms slide, tested.

Reading an orchestration question quickly

- ◆ **Count the non-Glue steps.** Zero means Glue workflows are viable.
- ◆ **Are there existing Airflow DAGs?** If so, MWAA, and now Serverless is an option.
- ◆ **Clock or event?** "Every night" is a schedule; "when it arrives" is a trigger.
- ◆ **Is the failure transient or permanent?** Retry the first, route the second.
- ◆ **What went unnoticed?** Silent partial success is the failure the exam cares about.

ON THE EXAM

Next: programming concepts. Lambda tuning, infrastructure as code, CI/CD, and the software engineering the exam expects.